

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего
образования «Самарский национальный исследовательский университет имени академика
С.П. Королева»
(Самарский университет)

Институт информатики и кибернетики

Кафедра информационных систем и технологий

ОТЧЕТ ПО ПРАКТИКЕ

Вид практики производственная
(учебная, производственная)

Тип практики научно-исследовательская работа
(в соответствии с ОПОП ВО)

Сроки прохождения практики: с 01.03.2025 по 30.05.2025
(в соответствии с календарным учебным графиком)

по направлению подготовки 09.03.01 Информатика и вычислительная техника
(уровень бакалавриата)
направленность (профиль) «Информационные системы»

Обучающийся группы № 6304-090301D _____ М.П. Федулов

Руководитель практики,
доцент кафедры ИСТ, к.т.н. _____ Я.В. Соловьева

Дата сдачи 30.05.2025

Дата защиты 30.05.2025

Оценка _____

Самара 2025

СОДЕРЖАНИЕ

1 Анализ технологий проектирования логических и физических моделей баз данных информационных систем.....	6
1.1 Общие принципы проектирования баз данных	7
1.2 Этапы моделирования: концептуальная, логическая, физическая модель	8
1.3 Обзор методологий	9
1.3.1 ER-моделирование.....	9
1.3.2 Нормализация и денормализация.....	11
1.4 Сравнительный анализ технологий и обоснование выбранной СУБД	12
2 Логическая и физическая модель БД разрабатываемой ИС	13
2.1 Описание предметной области	13
2.2 Построение логической модели	13
2.3 Построение физической модели.....	14
3 Анализ современных инструментальных средств разработки ПО для ИС ...	16
3.1 Требования к среде разработки ИС.....	16
3.2 Обзор современных инструментов	16
3.2.1 WebStorm	17
3.2.2 Microsoft Visual Studio.....	18
3.2.3 Visual Studio Code.....	18
3.3 Сравнение по критериям	19

3.4 Обоснование выбора конкретной IDE/среды.....	19
ЗАКЛЮЧЕНИЕ	21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	22
ПРИЛОЖЕНИЕ	23

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский университет
имени академика С.П. Королева»
(Самарский университет)

Институт информатики и кибернетики

Кафедра информационных систем и технологий

**Задание(я) для выполнения определенных видов работ, связанных с
будущей профессиональной деятельностью (сбор и анализ данных и
материалов, проведение исследований)**

Обучающемуся Федулову Михаилу Павловичу
группы 6304-090301D

Направление на практику оформлено приказом по университету
от 27.02.2025 г. № 114-ПР

на кафедру информационных систем и технологий
(наименование профильной организации или структурного подразделения университета)

Тема НИР: «Исследование подходов разработки, взаимодействия и
сборки микрофронтенд приложений»

Планируемые результаты освоения образовательной программы (компетенции)	Планируемые результаты практики	Содержание задания
ПК-1 Способен осуществлять проведение научно-исследовательских и опытно-конструкторских разработок по отдельным разделам темы ПК 1.1 Демонстрирует способность понимать, совершенствовать и применять современный инструментарий в ходе исследований в рамках профессиональной деятельности	Знать: современные инструментальные средства разработки программ. Уметь: применять современные инструментальные средства для разработки программ. Владеть: навыками разработки алгоритмов и программ.	Провести анализ современных инструментальных средств разработки программного обеспечения для информационных систем. Сделать обоснование выбора используемой среды разработки программного обеспечения информационной системы разработки, взаимодействия и сборки микрофронтенд приложений. Разработать основные алгоритмы и программное обеспечение информационной системы разработки, взаимодействия и сборки микрофронтенд приложений.

		Сделать описание основных алгоритмов и программного обеспечения.
ПК-7 Способен осуществлять выполнение работ и управление работами по созданию (модификации) и сопровождению информационных систем, автоматизирующих задачи организационного управления и бизнес-процессы. ПК-7.5 Разрабатывает базы данных информационных систем	Знать: технологии проектирования логических и физических моделей баз данных информационных систем. Уметь: применять технологии проектирования логических и физических моделей для разработки баз данных информационных систем. Владеть: навыками разработки баз данных информационных систем.	Провести анализ технологий проектирования логических и физических моделей баз данных информационных систем. Спроектировать базу данных информационной системы шифрования информации. Сделать описание логической и физической модели базы данных информационной системы разработки, взаимодействия и сборки микрофронтенд приложений.
ПК-7 Способен осуществлять выполнение работ и управление работами по созданию (модификации) и сопровождению информационных систем, автоматизирующих задачи организационного управления и бизнес-процессы. ПК-7.4 Проектирует и разрабатывает дизайн информационных систем.	Знать: технологии проектирования дизайна информационных систем. Уметь: применять технологии проектирования для разработки дизайна информационных систем. Владеть: навыками разработки дизайна информационных систем.	Провести анализ имеющихся технологий проектирования дизайна информационных систем и сделать обоснование выбора используемой технологии. Разработать дизайн информационной системы разработки, взаимодействия и сборки микрофронтенд приложений.

Дата выдачи задания 01.03.2025.

Срок представления на кафедру отчета о практике 30.05.2025.

Руководитель практики,
доцент кафедры ИСТ, к.т.н. _____ Я.В. Соловьева
(подпись)

Задание принял к исполнению
обучающийся группы № 6304-090301D _____ М.П. Федулов
(подпись)

1 Анализ технологий проектирования логических и физических моделей баз данных информационных систем

Одной из самых распространенных сфер применения вычислительной техники является создание различных автоматизированных систем. Среди них САПР (системы автоматизированного проектирования), АСУ (автоматизированные системы управления), АИС (автоматизированные информационные системы), АСНИ (автоматизированные системы научных исследований) и другие. Главное отличие этих систем – вид автоматизируемой человеческой деятельности. В САПР осуществляется автоматизация проектирования, в АСУ – управления (причем системы автоматизированного управления технологическими процессами называются АСУ ТП, управления производством – АСУП), в АИС – хранения и поиска данных, в АСНИ – научно-экспериментальных исследований.

В зависимости от способа хранения данных различают следующие виды баз данных:

- локальная база данных (централизованная) – база данных, хранящая данные в памяти одной вычислительной машины;
- распределенная база данных – база данных, хранящая данные в памяти различных ЭВМ вычислительной сети.

В зависимости от вида хранимой информации различают следующие виды баз данных:

- фактографические базы данных – базы данных, хранящие информацию в виде данных, отражающих фактические значения, или иначе текущее состояние предметной области;
- динамические базы данных – базы данных, хранящие данные и время их внесения или изменения, отображающие состояние предметной области в определенный момент времени;
- документальные базы данных – базы данных, хранящие информацию в виде документов, то есть в виде определенным образом

организованной информации, включая отчеты, монографии и другие виды документов;

- графические базы данных – базы данных, хранящие информацию в виде графических объектов, например видеоданные, «pictorial data base», «graphics based data base» и другие;
- интегрированные базы данных – базы данных, хранящие информацию в виде данных, документов, графических объектов в любой комбинации. [1]

1.1 Общие принципы проектирования баз данных

Проектирование базы данных – это итеративный процесс, включающий несколько ключевых этапов, каждый из которых имеет критическое значение для конечного результата.

Основные принципы проектирования:

1) Нормализация данных. Процесс нормализации заключается в разделении данных на несколько связанных таблиц для исключения избыточности и обеспечения целостности данных. Нормализация помогает избежать дублирования информации и снижает риск возникновения ошибок при обновлении данных;

2) Денормализация данных. В некоторых случаях денормализация может быть полезна для увеличения производительности. Это процесс объединения данных из нескольких таблиц в одну, что уменьшает количество соединений (JOIN) в запросах и ускоряет их выполнение;

3) Целостность данных. Необходимо обеспечить, чтобы данные в базе были согласованными и не содержали противоречий. Для этого используются механизмы обеспечения целостности, такие как первичные и внешние ключи, триггеры и ограничения;

4) Масштабируемость. Хорошо спроектированная база данных должна быть готова к росту объёмов данных. Масштабируемость включает не только физическое хранение данных, но и оптимизацию запросов для больших данных;

5) Безопасность данных. Для защиты конфиденциальной информации необходимо реализовать механизмы разграничения доступа, шифрования данных и аудита действий пользователей;

6) Управление транзакциями. Важно обеспечить правильное выполнение транзакций, чтобы избежать потерь данных при сбоях или ошибках. Системы управления транзакциями (ACID) гарантируют надёжное выполнение операций. [2]

1.2 Этапы моделирования: концептуальная, логическая, физическая модель

Моделирование баз данных включает три основных этапа: концептуальное, логическое и физическое моделирование. Эти этапы представляют собой последовательный процесс перехода от общего представления предметной области к конкретной реализации базы данных в выбранной СУБД.

Концептуальная модель данных включает в себя все основные сущности и связи, не содержит подробных сведений об атрибутах и часто используется на начальном этапе планирования.

Логическая модель данных — это расширение концептуальной модели данных. Она включает в себя все сущности, атрибуты, ключи и взаимосвязи, которые представляют бизнес-информацию и определяют бизнес-правила.

Физическая модель данных включает в себя все необходимые таблицы, столбцы, связи, свойства базы данных для физической реализации баз данных. Производительность базы данных, стратегия индексации, физическое хранилище и денормализация — важные параметры физической модели. [3]

1.3 Обзор методологий

1.3.1 ER-моделирование

Моделирование данных в ER-модели осуществляется при помощи базовых структурных элементов: сущность, атрибут и связь. Структурная составляющая, представляющая время, в модели в явном виде отсутствует. Время происхождения событий может быть представлено при помощи атрибутов.

Сущность позволяет представить в абстрактном виде материальные (личность, автомобиль, здание, судно) и не материальные (накладные, заказы, счета, акты) объекты прикладной области, сведения о которых необходимо хранить в базе данных. Каждая сущность является узловой точкой сбора информации о представляемом ею объекте. Различают такие понятия как тип сущности и экземпляр сущности. Тип сущности относится к множеству однородных объектов. Например, тип сущности «Личность» относится к множеству людей. Для того чтобы отличить один тип сущности от другого, каждому типу сущности ставится в соответствие уникальное наименование. Экземпляр сущности относится к конкретному элементу множества, образующего сущность. Например, «Петрова», «Федоров», «Иванов» являются экземплярами сущности «Личность». Для каждого типа сущности существует предикат, который проверяет принадлежность сущности данному типу. Если сущность принадлежит данному типу, то она имеет свойства, общие для всех экземпляров этого типа. Сущность может принадлежать более чем одному типу. Например, сущность, принадлежащая типу «мужчины» или «женщины», также принадлежит типу «личность». Это свойство рассматриваемых объектов может быть соответствующим образом учтено в модели. Для того чтобы задать сущность, необходимо сформулировать ей определение и выбрать ее четкое наименование, отражающее смысловое содержание определения. Сущность представляется в ER-диаграммах прямоугольником, содержащим наименование обозначаемого им типа сущности представлена на рисунке 1.

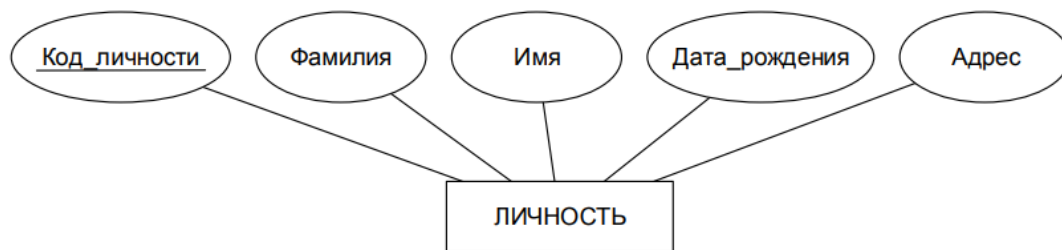


Рисунок 1 - Представление сущности и атрибутов

Атрибут является средством описания сущностей и связей. Он используется для определения того, какая информация должна быть собрана об описываемой им сущности или связи. Например, сущность «Личность» можно описать при помощи атрибутов «Код_личности», «Фамилия», «Имя», «Дата_рождения», «Адрес». Как видно из примера, атрибуты отличаются по их наименованиям. Наименование атрибута должно быть уникальным для конкретного типа сущности или связи. Различные типы сущностей или связей могут иметь атрибуты с одинаковыми наименованиями. Например, атрибут «Цвет» может быть определен для типов сущностей «Автомобили», «Обувь» и т. п. Однако на практике этого рекомендуется избегать, приписывая такому атрибуту имя сущности, например, «Цвет_автомобиля». Каждому атрибуту соответствует множество значений. Например, атрибуту «Цвет_автомобиля» соответствуют значения «красный», «зелёный», «голубой» и т. п. Каждому экземпляру сущности присваивается только одно значение атрибута. Это значение определяется в результате наблюдений или измерений. Атрибут характеризуется также ролью, которую он выполняет. Наиболее часто выполняемой ролью атрибута является описание свойств сущности. Например, атрибуты «Дата_рождения», «Рост» и «Адрес» описывают свойства сущности «Личность». Для идентификации сущности может быть использован один или несколько атрибутов. Например, атрибут «Код_личности» может быть самостоятельно использован для идентификации сущности «Личность». Совокупность атрибутов «Фамилия», «Имя», «Дата_рождения» может быть также использована для идентификации этой же сущности. Разработчик модели данных самостоятельно выбирает способ идентификации. В соответствии с

выполняемой ролью атрибуты подразделяются на описательные и идентифицирующие. Атрибуты могут выполнять и другие роли, однако, в настоящих указаниях они не рассматриваются. Для того чтобы задать атрибут в модели, необходимо присвоить ему наименование, определить смысловое содержание, множество допустимых значений и указать роль атрибута. В ER-диаграммах атрибут изображается овалом и соединяется ненаправленными ребрами с описываемой им сущностью или связью. Наименование проставляется внутри обозначения атрибута. Наименования идентифицирующих атрибутов также подчеркиваются [4].

1.3.2 Нормализация и денормализация

Нормализация – это процесс разбиения или декомпозиции исходного отношения на несколько отношений с целью устранения нежелательных функциональных зависимостей, приводящих к возникновению избыточности хранения информации и аномалиям добавления, удаления, обновления. Аппарат нормализации отношений разработан Коддом. В нем определены различные нормальные формы отношений. Каждая нормальная форма ограничивает типы допустимых функциональных зависимостей отношений. Кодд выделил три нормальные формы: 1, 2, 3НФ, а сегодня определены 4НФ, 5НФ.

1НФ – отношение находится в первой нормальной форме, если значения всех атрибутов атомарные, то есть значение атрибута не является множеством, диапазоном или повторяющейся группой.

2НФ – отношение находится во второй нормальной форме, если оно находится в первой нормальной форме и отсутствуют частичные функциональные зависимости атрибутов, не входящих в ключ от ключа (зависимости атрибутов, не входящих в ключ от части ключа). Иначе можно сказать, если каждый атрибут, не входящий в ключ, функционально полностью зависит от ключа. Анализируя определение второй нормальной формы, можно

сделать вывод, что отношение находится во второй нормальной форме, если в нем отсутствуют составные ключи. [1]

Денормализация – помещение избыточных данных туда, где они смогут принести максимальную пользу. Данное действие применяется уже к нормализованной БД. Для этого можно использовать дополнительные поля в уже существующих таблицах, добавлять новые таблицы или даже создавать новые экземпляры существующих таблиц. Логика в том, чтобы снизить время исполнения определенных запросов через упрощение доступа к данным или через создание таблиц с результатами отчетов, построенных на основании исходных данных. [5]

Преимущества:

- поддержка актуальности данных;
- повышение производительности;
- ускорение создания отчетов;
- предварительные вычисления часто запрашиваемых значений.

Недостатки:

- аномалии данных;
- повышение занимаемого места из-за дублирования данных;
- замедление других операций;
- увеличение объема кода.

1.4 Сравнительный анализ технологий и обоснование выбранной СУБД

Проектирование физической базы данных является важным этапом при разработке информационных систем, так как именно на этом уровне происходит непосредственная реализация логической модели в конкретной системе управления базами данных (СУБД). От правильного выбора СУБД зависит не только производительность и устойчивость всей системы, но и удобство сопровождения, масштабируемость, безопасность и стоимость эксплуатации.

Существует большое количество СУБД, различающихся по лицензии, архитектуре, поддерживаемым возможностям и совместимости с определёнными платформами. При выборе подходящего решения необходимо учитывать особенности проекта, объём обрабатываемых данных, предполагаемую нагрузку, требования к надёжности и безопасности, а также возможность интеграции с другими компонентами программной системы.

Наиболее известными реляционными базами данных являются Open Source проекты PostgreSQL, MySQL и SQLite, а также проприетарные решения Oracle, Microsoft SQL Server и IBM Db2Relational. [6]

Для нашей целей хорошо подойдет PostgreSQL, так как он обладает повышенной производительностью, масштабируемостью и надёжностью.

2 Логическая и физическая модель БД разрабатываемой ИС

2.1 Описание предметной области

Описание предметной области: автоматизированная система учета игр.

В базе данных хранятся сведения об играх, издателях и разработчиках.

Игра характеризуется ID игры, названием, платформой, жанром, годом релиза, краткой информацией о проекте, названием издателя, именем разработчика.

Издатель характеризуется ID издателя, названием издательства, годом основания, краткой информацией об издателе.

Разработчик характеризуется ID разработчика, именем разработчика, краткой информацией об разработчике.

2.2 Построение логической модели

Для построения логической модели использовалась программа ERwin Data Modeller. Логическая модель изображена на рисунке 1.

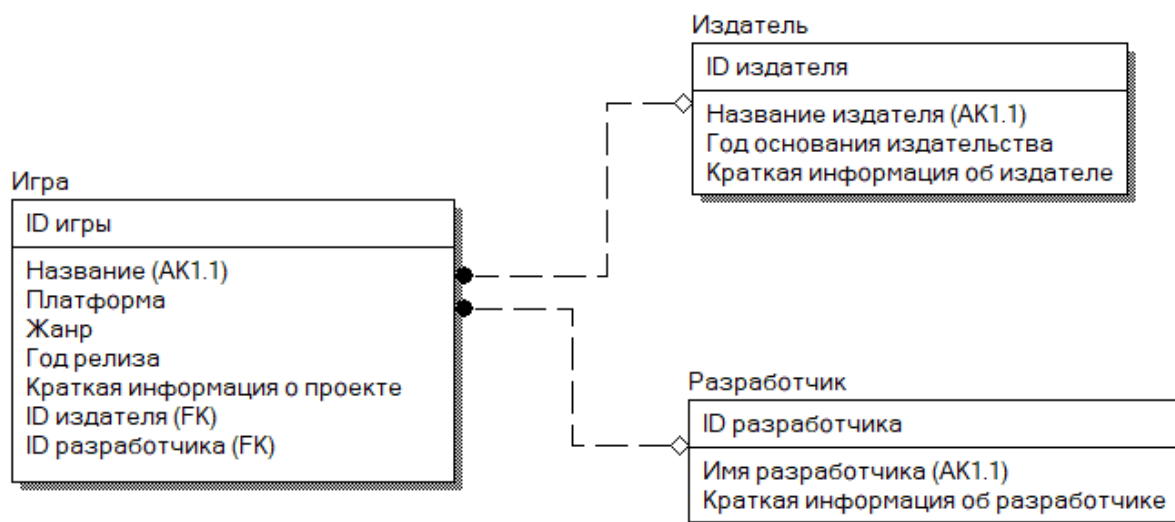


Рисунок 1 – Логическая схема базы данных

Данная модель обладает следующей схемой отношений:

Игра (ID игры, название, платформа, жанр, год релиза, краткая информация о проекте, ID издателя, ID разработчика);

Издатель (ID издателя, название издателя, год основания издательства, краткая информация об издателе);

Разработчик (ID разработчика, имя разработчика, краткая информация об разработчике).

2.3 Построение физической модели

Построение физической модели проводилось в программе PostgreSQL. Физическая модель изображена на рисунке 2.

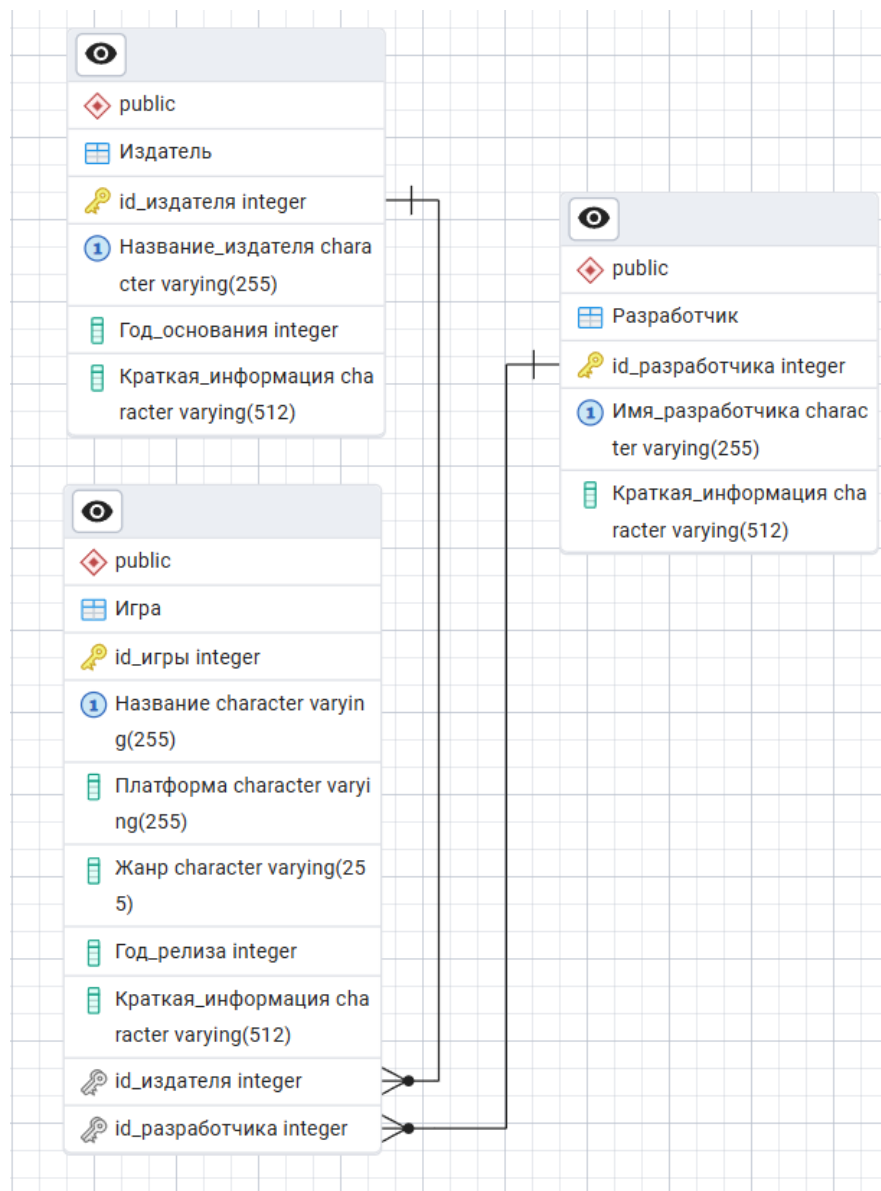


Рисунок 2 – Физическая схема базы данных

Физическая модель имеет 3 таблицы: «Игра», «Издатель», «Разработчик».

Таблица «Игра» имеет первичный ключ «id_игры» с типом данных integer, альтернативный ключ «Название» с типом данных varchar(255), и атрибуты «Платформа», «Жанр» с типом данных varchar(255), «Год_релиза» с типом Integer и «Краткая_информация» с типом varchar(512).

Таблица «Издатель» имеет первичный ключ «id_издателя» с типом данных integer, альтернативный ключ «Название_издателя» с типом данных varchar(255), и атрибуты «Год_основания» с типом Integer и «Краткая_информация» с типом varchar(512).

Таблица «Разработчик» имеет первичный ключ «id_издателя» с типом данных integer, альтернативный ключ «Имя_разработчика» с типом данных varchar(255), и «Краткая_информация» с типом varchar(512).

3 Анализ современных инструментальных средств разработки ПО для ИС

3.1 Требования к среде разработки ИС

Для создания автоматизированной системы учета игр, лучше всего использовать инструменты, которые обеспечивают:

- высокую скорость работы;
- простота использования;
- возможность использования различных расширений.

Найдём подходящую среду разработки и код программирования удовлетворяющие требованиям разработки.

3.2 Обзор современных инструментов

IDE (среда интегрированной разработки) — это программное обеспечение, которое обеспечивает все необходимые инструменты для разработки программного обеспечения в одном месте.

Инструменты разработки программного обеспечения предназначены для повышения эффективности и упрощения процесса разработки за счет автоматизации задач и предоставления удобных пользовательских интерфейсов. Эти инструменты позволяют разработчикам писать код с меньшим количеством ошибок, использовать функции перетаскивания и создания приложений с широким спектром функций.

Современные инструменты разработки программного обеспечения вышли за рамки традиционных интегрированных сред разработки (IDE), в которых программисты пишут код вручную. Теперь они предлагают широкий спектр функций, включая:

- редакторы кода с подсветкой синтаксиса, автозаполнением и навигацией по коду;
- отладчики для поиска и исправления ошибок;
- инструменты контроля версий для управления изменениями кода;
- инструменты тестирования для проверки работоспособности программного обеспечения;
- генераторы кода для автоматического создания кода на основе определенных спецификаций;
- библиотеки и фреймворки для повторного использования кода и ускорения разработки.

Выбор инструментов программирования и языков программирования зависит от множества различных факторов. Для начала стоит учитывать собственные требования к IDE, чтобы набор её функций соответствовал выполняемым задачам. Инструментарий и возможности выбранной IDE – второй важный аспект, так как инструменты платформы должны иметь необходимые и современные возможности, такие как совместимость с различным оборудованием, гибкость (поддержка различных языков программирования, кроссплатформенная разработка и др.) и не занимать объёмное количество ресурсов.

Рассмотрим самые оптимальные инструменты для разработки программного обеспечения, и выберем оптимальный вариант.

3.2.1 WebStorm

WebStorm – это мощная IDE от JetBrains, предназначенная для разработки веб-приложений.

Достоинства:

- удобство разработки;
- глубокая интеграция с фреймворками;
- подробная документация;

- плагины и расширяемость.

Недостатки:

- только платная версия;
- высокие системные требования.

3.2.2 Microsoft Visual Studio

Visual Studio — это мощное средство разработчика, которое можно использовать для выполнения всего цикла разработки в одном месте. Её можно использовать для записи, редактирования, отладки и сборки кода, а затем развертывания приложения. Помимо редактирования и отладки кода Visual Studio включает компиляторы, средства завершения кода, управление версиями, расширения и многое другое, чтобы улучшить каждый этап процесса разработки программного обеспечения.

Достоинства:

- кастомизация рабочей панели;
- автодополнение IntelliSense;
- поддержка разделенного экрана;
- большое число расширений, которое постоянно пополняется.

Недостатки:

- зависимость от экосистемы Microsoft;
- занимает много места на диске;
- требовательность к ресурсам компьютера;
- сложность в освоении.

3.2.3 Visual Studio Code

Visual Studio Code – это один из наиболее популярных редакторов кода, разработанный корпорацией Microsoft. Он распространяется в бесплатном доступе и поддерживается всеми актуальными операционными системами: Windows, Linux и macOS. VS Code представляет собой обычный текстовый

редактор с возможностью подключения различных плагинов, что дает возможность работать со всевозможными языками программирования для разработки любого IT-продукта.

Достоинства:

- ряд полезных настроек для автоматизации рабочего процесса;
- простота освоения;
- малый вес;
- огромное количество плагинов, которые увеличивают функционал;
- кроссплатформенность.

Недостатки:

- медленная работа;
- менее надёжный для больших проектов.

3.3 Сравнение по критериям

Таблица 1 – Критерии сравнения сред разработки

Критерий	WebStorm	Visual Studio	VS Code
Поддержка языков	JavaScript, TypeScript, HTML, CSS	C++, C#, Python	Все через плагины
Интеграция с БД	Через плагины	Да (SQL Server и др.)	Через расширения
UI-дизайнеры	Live Preview, CSS Preview, интеграция с Figma	Есть (WinForms, WPF)	Ограничено

3.4 Обоснование выбора конкретной IDE/среды

Оценив все преимущества всех сред разработки, я пришёл к выводу, что лучшей средой разработки будет VS Code из-за поддержки всех языков

программирования (среди которых есть необходимые JavaScript и TypeScript) возможности подключать различные расширения для отладки и быстрого написания кода.

ЗАКЛЮЧЕНИЕ

В процессе выполнения научно-исследовательской работы были освоены индикаторы ПК-1.1, ПК-7.5 компетенций ПК-1, ПК-7, и решены все поставленные задачи:

- был проведен анализ технологий проектирования логических и физических моделей баз данных информационных систем;

- была спроектирована база данных информационной системы разработки, взаимодействия и сборки микрофронтенд приложений;

- было сделано описание логической и физической модели базы данных информационной системы разработки, взаимодействия и сборки микрофронтенд приложений;

- был проведен анализ современных инструментальных средств разработки программного обеспечения для информационных систем;

- было сделано обоснование выбора используемой среды разработки программного обеспечения информационной системы разработки, взаимодействия и сборки микрофронтенд приложений;

- были разработаны основные алгоритмы и программное обеспечение информационной системы разработки, взаимодействия и сборки микрофронтенд приложений;

- было сделано описание основных алгоритмов и программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Чигарина Е.И. Ч586 Базы данных: учеб. пособие [Текст] / Е.И. Чигарина. – Самара: Изд-во СГАУ, 2015. – 208 с. (Дата обращения: 29.05.2025)
2. От анализа до внедрения: полный цикл разработки баз данных [Электронный ресурс] // URL: <https://kasatkin.io/blog/ru-how-to-design-an-efficient-database> (Дата обращения: 29.05.2025)
3. Моделирование данных: обзор [Электронный ресурс] // URL: <https://habr.com/ru/articles/556790/> (Дата обращения: 29.05.2025)
4. Инфологическое моделирование [Электронный ресурс] // URL: <https://www.dvfu.ru/upload/medialibrary/a43/Сухомлинов%20А.И.%20Инфологическое%20моделирование.pdf> (Дата обращения: 29.05.2025)
5. Зачем нужна денормализация баз данных, и когда ее использовать [Электронный ресурс] // URL: <https://habr.com/ru/companies/latara/articles/281262/> (Дата обращения: 29.05.2025)
6. Виды баз данных. Большой обзор типов СУБД [Электронный ресурс] // URL: <https://habr.com/ru/companies/amvera/articles/754702/> (Дата обращения: 29.05.2025)

ПРИЛОЖЕНИЕ

Код для создания таблиц в PostgreSQL

-- Создаем таблицу "Издатель"

CREATE TABLE Издатель (

 ID_издателя INTEGER PRIMARY KEY, -- Уникальный
идентификатор

 Название_издателя VARCHAR(255) NOT NULL UNIQUE, --
Уникальное название издателя

 Год_основания INTEGER, -- Год основания издательства

 Краткая_информация VARCHAR(512) -- Краткая информация об
издателе

);

-- Создаем таблицу "Разработчик"

CREATE TABLE Разработчик (

 ID_разработчика INTEGER PRIMARY KEY, -- Уникальный
идентификатор

 Имя_разработчика VARCHAR(255) NOT NULL UNIQUE, --
Уникальное имя разработчика

 Краткая_информация VARCHAR(512) -- Краткая информация
о разработчике

);

-- Создаем таблицу "Игра"

CREATE TABLE Игра (

 ID_игры INTEGER PRIMARY KEY, -- Уникальный
идентификатор

 Название VARCHAR(255) NOT NULL UNIQUE, -- Уникальное
название игры

 Платформа VARCHAR(255), -- Платформа игры

```

        Жанр VARCHAR(255),           -- Жанр игры
        Год_релиза INTEGER,          -- Год релиза игры
        Краткая_информация VARCHAR(512), -- Краткая информация о
        проекте
        ID_издателя INTEGER REFERENCES Издатель(ID_издателя), --
        Внешний ключ на таблицу "Издатель"
        ID_разработчика INTEGER REFERENCES
        Разработчик(ID_разработчика) -- Внешний ключ на таблицу
        "Разработчик"
    );

```